

Facial Emotion Recognition: Technical Report

1. Introduction

Problem Statement

In this project, I tried to make an AI that can detect emotions from faces in images. The AI needs to tell if someone is angry, scared, happy, neutral, sad or surprised.

Motivation

This technology is useful for many things like:

- Making computers understand human feelings better
- Checking mental health
- Seeing how customers feel
- Making education better
- Safety in cars (checking if drivers are tired)

Approach

I used deep learning with CNN (Convolutional Neural Networks) and PyTorch to build my model. My goal was to make it recognize six emotions from face pictures.

Repository

<https://github.com/DaanHoeven/facial-emotion-recognition>

2. Data

Dataset Description

I used the FER-2013 dataset from Kaggle. It was easy to find because it's one of the most popular datasets for this kind of project. It has face pictures with emotion labels.

Preprocessing Steps

I did these things to prepare the images:

1. Made all images grayscale
2. Resized them to 56×56 pixels
3. Used data augmentation (flipping images, rotating them a bit)
4. Normalized pixel values to [-1, 1]

Challenges with the Dataset

Working with this dataset had some problems:

1. Some emotions have more pictures than others
2. Some emotions look similar
3. Images are small and not very clear
4. Faces are in different positions

3. Model & Methods

Model Architecture

I built a CNN model with:

- 4 convolutional blocks with more filters in each (64→128→512→512)
- Batch normalization after each layer to make training more stable
- Dropout (p=0.25) to stop overfitting
- Two fully connected layers (256→512)
- Output layer with 6 units (one for each emotion)

Training Method

For training my model I used:

1. Cross-Entropy Loss function
2. Adam optimizer starting with learning rate 0.0001
3. OneCycleLR scheduler with max_lr=0.001
4. Batch size of 64
5. Maximum 500 epochs
6. Early stopping when accuracy reached 83% (my goal was 83%+ on val_acc and 83%+ on train_acc)

7. Saved the best model based on validation accuracy

4. Results & Evaluation

Training Progress

I trained the model for different numbers of epochs:

Epochs	Validation Accuracy
100	78.27%
200	81.41%
292	85.98%

Per-Class Performance

Emotion	100 Epochs	200 Epochs	292 Epochs
Angry	66.35%	73.44%	81.25%
Fear	65.13%	65.82%	76.33%
Happy	93.10%	94.74%	96.27%
Neutral	76.97%	82.89%	83.14%
Sad	71.47%	73.84%	80.16%
Surprise	87.20%	88.96%	93.10%

Main Findings

1. My model is really good at finding "Happy" and "Surprise" emotions (more than 93% correct)
2. "Fear" was the hardest to detect (only 76.33% correct in best model)
3. Training for more epochs generally improved performance, with significant gains between 200 and 292 epochs
4. Some emotions got easier to detect with more training, but some didn't improve much

5. What I Did

My Work

1. Got the FER-2013 dataset and prepared it
2. Made a custom CNN model
3. Created the training process
4. Tried different settings to get better results
5. Tested the model performance for each emotion
6. Made a function to use the model on new images

Things I Used

1. FER-2013 dataset from Kaggle
2. Looked at other GitHub projects for ideas (but didn't copy code)
3. PyTorch documentation for help

6. Problems & Future Improvements

Problems I Had

1. Some emotions were harder to detect than others
2. Training took a long time
3. The images were small and not very clear

Future Work

I could make this project better by:

1. Trying different model architectures like ResNet or EfficientNet
2. Getting more training data, especially for emotions that are hard to detect
3. Making a real-time system that works with webcams (video support)
4. Making the model smaller and faster
5. Adding more emotion types

Conclusion

My facial emotion recognition project worked pretty well. The final model got 85.98% accuracy after 292 epochs, exceeding my goal of 83%+ accuracy on both training and validation sets. It's very good at detecting "happy" and "surprise" but still has problems with "fear". Currently the model works on static images but not yet on videos. In the future, I could make it better with more data, better models, and more training.